3-D Representation

- Graphics scenes can contain many different kinds of objects and material surfaces such as Trees,Flowers,Clouds,rocks,water ,bricks,steel ,plastic and cloth……etc
- No single method can be used to describe objects that will include all features of those different materials.
- To produce realistic display of scenes, we need to use representations that accurately model object characteristics.
  Eg polyhedron(a shape formed by many polygon) and ellipsoids can be represented by polygon and quadric surfaces.
- Engineering objects like aircraft wings,gears etc can be represented by spline surfaces.
- Medical CT images can be represented by octree representation.

Prepared by: ER
sandeep kumar poudel

Representation scheme is divided into two broad categories

a) Boundary representation-

- **Boundary Representation (B-Rep)** is a widely used method in computer graphics, CAD/CAM, and solid modeling to represent **3D objects by explicitly defining their boundaries** rather than their interior.
- Describe object in terms of its surface boundary eg; vertices, edges and faces.
- B-rep for simple polyhedron satisfies Euler's formula V-E+F = 2

b) Space partitioning representation-

- **Space Partitioning Representation** is a method of representing 3D objects by **dividing the entire space into smaller, non-overlapping regions (cells)** and describing which parts of space are occupied by the object.
- A solid is represented as a collection of adjoining non- intersecting solids.
- Used to describe interior properties of a solid eg; octree representation.

3D Object Representation - Polygon Surfaces

Polygon Surface Representation is one of the most common methods for representing 3D objects in computer graphics.

In this approach, the surface of a 3D object is approximated using a collection of **flat polygonal faces**, usually **triangles or quadrilaterals**.

The most commonly used boundary representation for a 3D object is a set of polygon surfaces that enclose the object interior.

Three ways to represent the polygon surfaces

1. Polygon Tables
2. Plane Equations
3. Polygon meshes

1 Polygon Tables:

- Refers to the tabular data structures used for organizing information about 3 D objects.
- Representation of vertex coordinates, edges and other property of polygon into table form is called polygon table.
- Polygon data tables can be organized into two groups:
  - **Geometric table**
  - **Attributes table.**

Geometric table contains vertex coordinate and the other parameter which specify geometry of polygon.

Convenient way to represent geometric table into three different table namely vertex table, edge table, and polygon table

(i) Vertex table: It stores co-ordinate values for each vertex of the object.

(ii) Edge table: It stores the edge information of polygon.

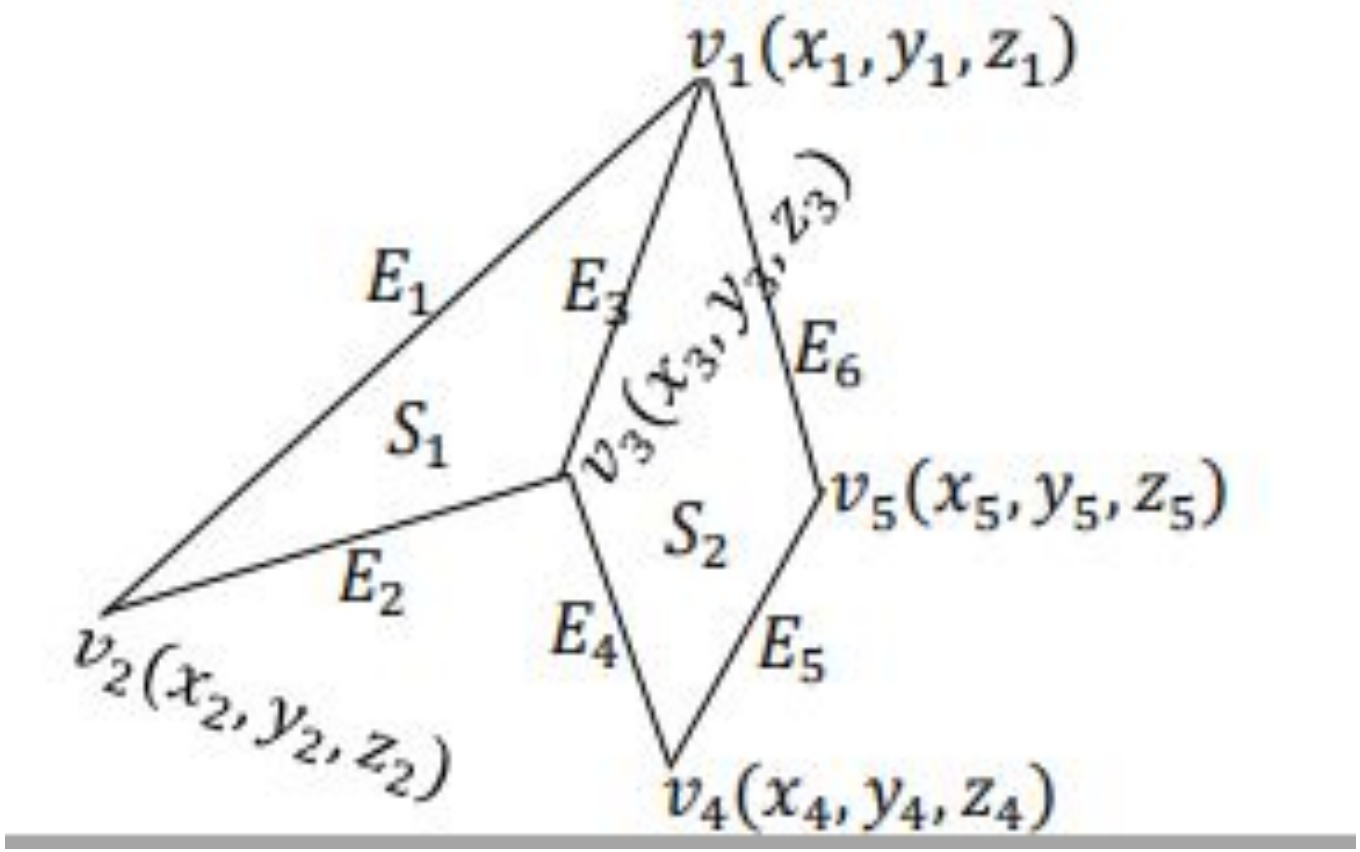
(iii) Surface table: It stores the number of surfaces present in the polygon.

Attribute table: It gives attribute information for an object (Degree of transparency, surface reflectivity etc.)

Some consistency checks of the geometric data table :

- Every vertex is listed as an endpoint for at least 2 edges .
- Every edge is part of at least one polygon.
- Every polygon is closed.
- Each polygon has at least one shared edge.

.


$v_1(x_1, y_1, z_1)$
E_1
E_3
$v_3(x_3, y_3, z_3)$
E_6
S_1
$v_5(x_5, y_5, z_5)$
S_2
E_2
E_4
E_5
$v_2(x_2, y_2, z_2)$
$v_4(x_4, y_4, z_4)$

Vertex table

$v_1: x_1, y_1, z_1$
$v_2: x_2, y_2, z_2$
$v_3: x_3, y_3, z_3$
$v_4: x_4, y_4, z_4$
$v_5: x_5, y_5, z_5$

Edge table

$E_1: v_1, v_2$
$E_2: v_2, v_3$
$E_3: v_3, v_1$
$E_4: v_3, v_4$
$E_5: v_4, v_5$
$E_6: v_5, v_1$

Surface table

$S_1: E_1, E_2, E_3$
$S_2: E_3, E_4, E_5, E_6$

The object can be displayed efficiently by using data from tables and processing them for surface rendering and visible surface determination.

Attribute table: It gives attribute information for an object (Degree of transparency, surface reflectivity etc.).

The above three table also include the polygon attribute according to their pointer information

Extra information can be added to the data tables for faster information extraction for instance,edge table can be expanded to include forward points into the polygon table so that common edges between polygons can be identified more rapidly

$E_1: v_1, v_2, S_1$
$E_2: v_2, v_3, S_1$
$E_3: v_3, v_1, S_1, S_2$
$E_4: v_3, v_4, S_2$
$E_5: v_4, v_5, S_2$
$E_6: v_5, v_1, S_2$

2. Plane Equations

-For producing display of 3 D objects we must process the input data representation for the object through several procedures.

- For this processing we sometimes need to find orientation and it can be obtained by vertex coordinate values and the equation of polygon plane

Plane Equation

The general equation of a plane is:

$Ax+By+Cz+D=0$

Where:

- **A, B, C** → Components of the **surface normal vector**
- **D** → Plane constant
- **(x, y, z)** → Any point on the plane
- the value of coefficient must be known to represent the 3D object.

The value of A, B, C & D can be obtained by solving a set of three plane equation using coordinate of three non-collinear point on plane. Let us assume that three vertices of plane are ($x1$, $y1$, $z1$), ($x2$, $y2$, $z2$), ($x3$, $y3$, $z3$). Then,

$$Ax1 + By1 + Cz1 + D = 0$$

$$Ax2 + By2 + Cz2 + D = 0$$

$$Ax3 + By3 + Cz3 + D = 0$$

For solving ABCD consider three successive polygon vertices (x_1, y_1, z_1), (x_2, y_2, z_2), (x_3, y_3, z_3)
So equation (i) is modified to,

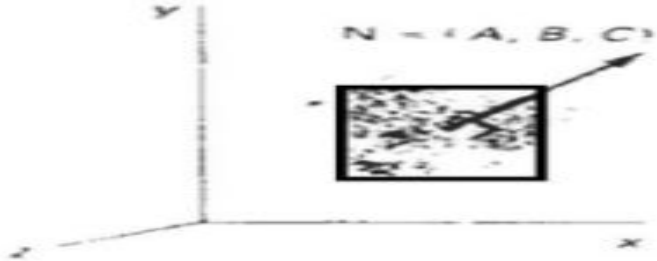
$$\frac{A}{D} x_k + \frac{B}{D} y_k + \frac{C}{D} z_k = -1 \quad \text{.....(ii)} \qquad k = 1,2,3$$

Using Cramers rule,

$$A = \begin{vmatrix} 1 & y_1 & z_1 \\ 1 & y_2 & z_2 \\ 1 & y_3 & z_3 \end{vmatrix} \qquad B = \begin{vmatrix} x_1 & 1 & z_1 \\ x_2 & 1 & z_2 \\ x_3 & 1 & z_3 \end{vmatrix} \qquad C = \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix} \qquad D = \begin{vmatrix} x_1 & y_1 & z_1 \\ x_2 & y_2 & z_2 \\ x_3 & y_3 & z_3 \end{vmatrix}$$

Expanding Determinants,

$$A = y_1 (z_2 - z_3) + y_2 (z_3 - z_1) + y_3 (z_1 - z_2)$$
$$B = z_1 (x_2 - x_3) + z_2 (x_3 - x_1) + z_3 (x_1 - x_2)$$
$$C = x_1 (y_2 - y_3) + x_2 (y_3 - y_1) + x_3 (y_1 - y_2)$$
$$D = -x_1 (y_2 z_3 - y_3 z_2) - x_2 (y_3 z_1 - y_1 z_3) - x_3 (y_1 z_2 - y_2 z_1)$$



The vector N, normal to the surface of a plane described by equation Ax + By + Cz + D =0 has Cartesian Components (A,B,C)

Plane equations are used to identify the position of spatial points relative to the plane surfaces of an object. For any point (x,y,z) not on plane with parameters ABCD we have

$$Ax + By + Cz + D = 0$$

We can identify the point as either inside or outside the plane surface according to the sign(+ or -) of

$$Ax + By + Cz + D$$

If $Ax + By + Cz + D < 0$ then the point (x,y,z) is inside the surface
If $Ax + By + Cz + D > 0$ then the point (x,y,z) is outside the surface

3.Polygon mesh

- Polygon mesh is a collection of edges, vertices and faces that defines the shape of the polyhedral object in 3D computer graphics and solid modeling.
- An edge can be shared by two or more polygons.
- Vertex is shared by at least two edges.

The polygon mesh can be represented by three ways-

Explicit representation

Pointers to a vertex list

Pointers to an edge list

Fig: a triangle strip formed with 11 triangles

Connecting 13 vertices.

Fig : a quadrilateral mesh connecting 12

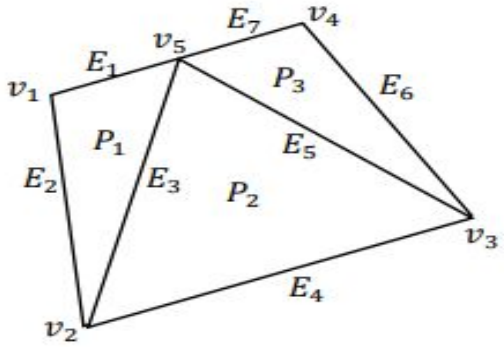
Quadrilateral.

In **Explicit representation**, each polygon is represented by a list of vertex co-ordinates in order in the memory.

P = ((x1, y1, z1), (x2, y2, z2), … … . . , (xn, yn, zn))

- It process fast but requires more memory for storing.

In **Pointers to a vertex list**, each vertex is stored just once, in vertex list V = (v1, v2, … … …. , vn) E.g. A polygon made up of vertices 3, 5, 7, 10 in vertex list be represented as P1 = {3,5,7,10}



Representing polygon mesh with each polygon as vertex list.

$P1 = \{v1, v2, v5\}$

$P2 = \{v2, v3, v5\}$

$P3 = \{v3, v4, v5\}$

In **Pointers to an edge list**, we have vertex list V, represent the polygon as a list of pointers to an edge list.

Each edge in edge list points to the two vertices in the vertex list. Also to one or two polygon, the edge belongs.

Hence, we describe polygon as $P = (E1, E2, \ldots \ldots \ldots . . , En)$ and an edge as $E = (v1, v2, P1, P2)$ Here if edge belongs to only one polygon.

For the mesh given above,

$V = \{v1, v2, v3, v4, v5\} = \{((x1, y1, z1), \ldots \ldots\ldots, (x5, y5, z5)\}$

$E1 = (v1, v5, P1)$

$E6 = (v3, v4, P3)$ $E2 = (v1, v2, P1)$

$E7 = (v4, v5, P3)$ $E3 = (v2, v5, P1, P2)$

$P1 = (E1, E2, E3)$ $E4 = (v2, v3, P2)$

$P2 = (E3, E4, E5)$

$E5 = (v3, v5, P1, P3)$ $P3 = (E5, E6, E7)$

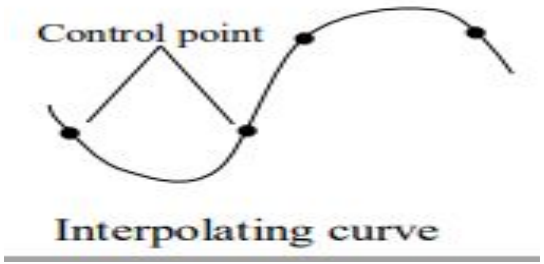
Spline Representation

- Spline are smooth curves passing through a set of given points.
- A Spline is a flexible stirps used to produce smooth curve through a designated set of points.
- A curve drawn with these set of points is spline curve.
- Spline curves are used to model 3D object surface shape smoothly.
- In computer graphics , continuous curve that are formed with polynomial section with certain boundary conditions are called spline curve.
- Mathematically spline are described as piecewise cubic polynomial functions.
- Spline is used in graphics application to design and digitize drawings for storage in computer and to specify animation path for the objects or the camera in the scene.

Two types of spline curve:

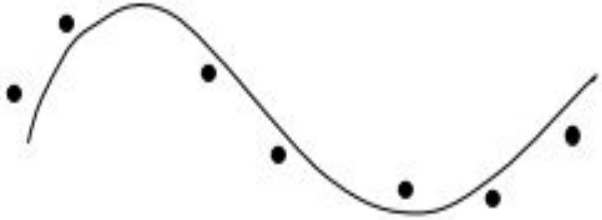
a. **Interpolating curve:** A curve that actually passes through each control point.

- Interpolation curves are commonly used to digitize drawing or to specify animation paths.



b. **Approximating curve:** A curve that passes near to control point but not necessary through them.

- Approximation curves are uses as design tools to structure object surfaces.



Approximating curve

- Spline curve also used for design of automobile bodies, spacecraft, specification of animation path, home appliance etc.
- The three degree polynomial known as cubic polynomial is typically used for constructing smooth curve in computer graphics because of following reasons:

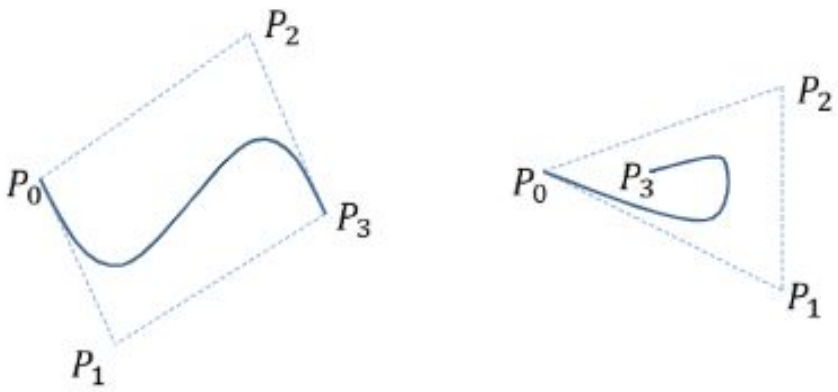
  a. It is lowest degree polynomial that can support an inflection (a point at which curve crosses its tangent i.e. curve changes from concave

  b. The curves are smooth like this and not jumpy like this

Convex Hull

The convex polygon boundaries that enclose a set of control points is called the **convex hull**

The shape of the convex hull is to imagine a rubber band stretched around the position of the control points so that each control point is either on the perimeter of the hull or inside it.



Parametric Continuity Condition

- For smooth transition from one curve section to next curve section we put various continuity conditions at connection points.
- Let parametric coordinate function as ,

$x=x(u)$, $y=y(u)$, $z=z(u)$ where $u1<<u<<u2$

- Then **zero order parametric continuity (c^0)** means simply curves meets.
- It means last point of first curve section & first points of second curve section are same.

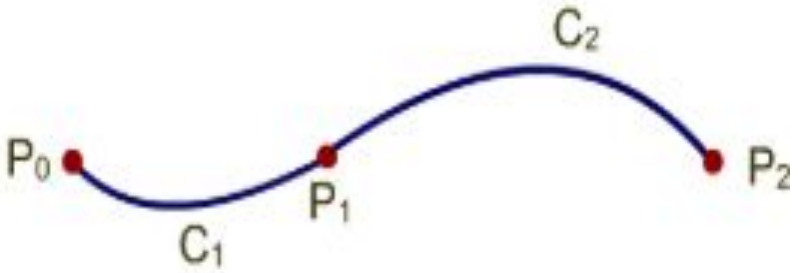
- **First order parametric continuity (c^1)** means first parametric derivatives are same for both curve section at intersection points.
- **Second order parametric continuity (c^2)** means both the first & second parametric derivative of two curve section are same at intersection.
- Higher order parametric continuity is can be obtain similarly.
- First order continuity is often sufficient for general application.
- Some graphics package like cad requires second order continuity for accuracy.

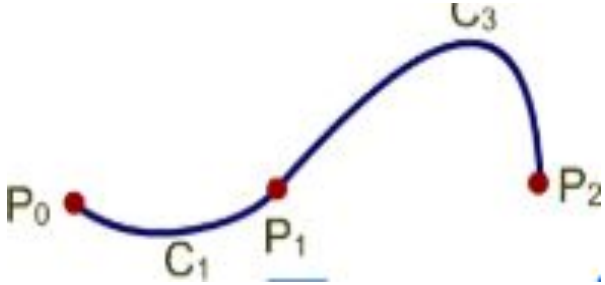
Geometric continuity condition

- Another method for joining two successive curve sections, is to specify condition for geometric continuity.
- **Zero order geometric continuity (g^0)** is same as parametric zero order continuity that two curve section meets.
- **First order geometric continuity (g^1)** means that the parametric first derivatives are proportional at the intersection of two successive sections.
- It is not necessary that its magnitude will be equal.
- In other word, tangent direction must be same magnitude may differ.

Contd..

- **Second order geometric continuity (g^2)** means that the both parametric first & second derivatives are proportional at the intersection of two successive sections.
- It is not necessary that its magnitude will be equal.
- With geometric continuity curve is pulled towards the section with the greater tangent vector.





Spline specifications

- There are three methods to specifying a particular spline representation:

  a) Boundary condition: We can state the set of boundary conditions that are imposed on the spline.

$$x(u) = a_x u^3 + b_x u^2 + c_x u + d_x, \qquad 0 \le u \le 1$$

  Boundary condition for this curve can be set for x(0), x(1), x'(0) & x'(1). These four conditions are sufficient to determine the values of four coefficient ax, bx, cx & dx.

  b) Characterizing matrix: We can state the matrix that characterizes the spline. From the boundary condition, the characterizing matrix for spline is:

$$x(u) = [u^3 \quad u^2 \quad u \quad 1] \begin{bmatrix} a_x \\ b_x \\ c_x \\ d_x \end{bmatrix}$$

$$= U.C$$

c) Blending Function: We can state the set of blending functions (or basis functions) that determine how specific geometric constraints on the curve are combined to calculate positions along the curve path.

$$x(u) = \sum_{k=0}^{3} g_k \cdot BF_k(u)$$

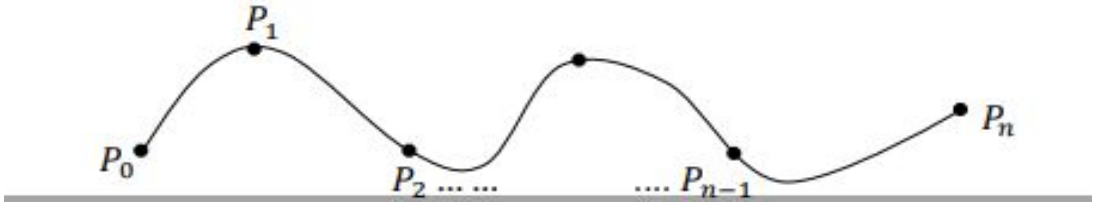
g_k = Geometric constrain parameter
$BF_k(u)$=Polynomial blending function

Cubic spline:

- It is used to set up path for object motions or to provide a representation for an existing object or drawings.
- Compared to higher-order polynomials, cubic splines requires less calculation and memory and they are more stable. Compared to lower-order polynomials, cubic splines are more flexible for modeling arbitrary curve shapes.
- Cubic interpolation spline is obtained by fitting the input points with a piecewise cubic polynomial curve that passes through every control points.

Suppose we have n+1 control points having co-ordinates

$$P_k = (x_k, y_k, z_k) \qquad K = 0, 1, 2, 3, \dots\dots\dots, n$$



A parametric cubic polynomial that is to be fitted between each pair of control points have following equations:

$x(u) = axu\ 3 + bxu\ 2 + cxu + dx$

$y(u) = ayu\ 3 + byu\ 2 + cyu + dy \qquad (0 \le u \le 1)$

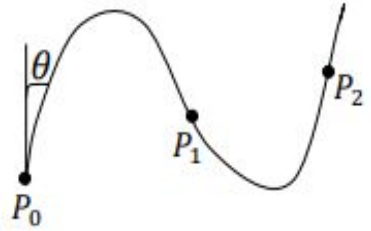
$z(u) = azu\ 3 + bzu\ 2 + czu + dz$

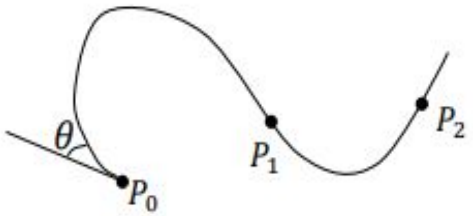
We need to determine the values of the four coefficients a, b, c, and d in the polynomial representation for each of the n curve section. We do this by setting enough boundary conditions at the “joints” between curve sections we can obtain numerical values for all the coefficients.

- Cubic splines are more flexible for modeling arbitrary curve shapes.

Hermite Interpolation (Hermite curve):

- It is an interpolating piecewise cubic polynomial with a specified tangent at each control point.





- If we change the control point at $P0$, then the curve will also change, so that angle θ between $P0$ and tangent at $P0$ will remain constant.
- It has **local control** over the curve i.e. each curve section depend on its end point only.
- The vector equivalence of Hermite curve is

p(u) = au 3 + bu 2 + cu + d ..... (i) 0<u<1

Where, x component of $P(u)$ is

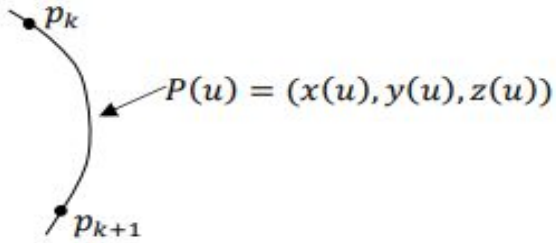
$$x(u) = a_x u^3 + b_x u^2 + c_x u + d_x$$

Similarly, y and z component

$$y(u) = a_y u^3 + b_y u^2 + c_y u + d_y$$

$$z(u) = a_z u^3 + b_z u^2 + c_z u + d_z$$

Let $P(u)$ denotes the parametric cubic point function for the curve section between control point p_k & p_{k+1}.



At p_k, u=0

$\therefore P(0) = p_k$

At p_{k+1}, u=1

$\therefore P(1) = p_{k+1}$

Also let Dp_k & Dp_{k+1} denote the slope at p_k & p_{k+1}.

$\therefore\ P'(0) = Dp_k$

$P'(1) = Dp_{k+1}$

Hence boundary condition for Hermite curve

$P(0) = p_k$

$P(1) = p_{k+1}$ ..... (ii)

$P'(0) = Dp_k$

$P'(1) = Dp_{k+1}$

Bezier curves and surfaces

- This spline approximation method was developed by the French engineer Pierre Bezier for the use of Renault automobile bodies.
- Bezier splines have a number of properties that make them highly useful and convenient for curve surface design.
- They are also easy to implement .
- It is widely available in various CAD systems,in general graphics packages and in assorted drawing and painting packages.

Bezier curves

- This method employs control points and produce an approximating curve.
- Bezier curve can be specified with boundary conditions,with characterizing matrix and blending function.
- Number of control points and their relative position gives degree of the Bezier polynomials.
- Most convenient method is to specify Bezier curve with blending function.

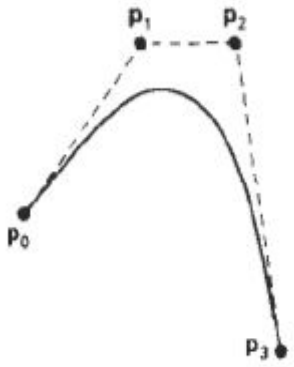


Figure : Bezier Curve

In above figure,

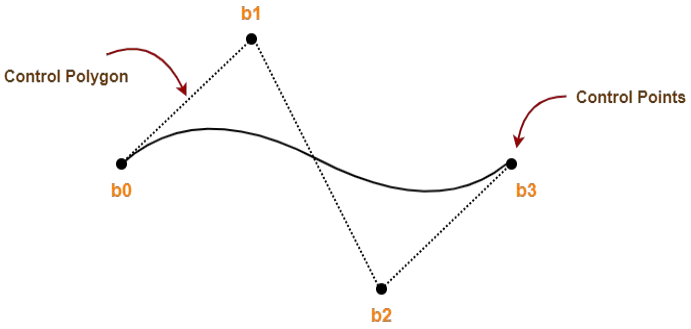
- This Bezier curve is defined by a set of control points p0,p1,p2 and p3.
- Point p0 and p3 are ends of the curve.
- Point p1 and p2 determine the shape of the curve.

Bezier Curve may be defined as-

Bezier Curve is parametric curve defined by a set of control points.

- Two points are ends of the curve.
- Other points determine the shape of the curve.

Bezier Curve Example-



Bezier Curve Example

Here,

- This bezier curve is defined by a set of control points b_0, b_1, b_2 and b_3.
- Points b_0 and b_3 are ends of the curve.
- Points b_1 and b_2 determine the shape of the curve.

Derivation for Equation of Bezier curve:

General Bezier curve for (n+1) control point, denoted as pk = (xk, yk, zk) with 'k' varying from 0 to n is given by

$$P(u) = \sum_{k=0}^{n} p_k BEZ_{k,n}(u), \qquad 0 \le u \le 1 \quad \ldots\ldots\ldots\ldots (i)$$

Where, $P(u)$ is a point on Bezier Curve.

p_k is a control point.

$BEZ_{k,n}(u)$ is a Bezier blending function also known as *Bernstein Polynomial.*

Bezier blending function is defined as

$$BEZ_{k,n}(u) = C(n,k)u^k(1-u)^{n-k}$$

Where, $C(n,k) = \frac{n!}{K!(n-k)!}$

Individual x, y, z coordinates an a Bezier curve is given by,

$$x(u) = \sum_{k=0}^{n} x_k BEZ_{k,n}(u)$$

$$y(u) = \sum_{k=0}^{n} y_k BEZ_{k,n}(u)$$

$$z(u) = \sum_{k=0}^{n} z_k BEZ_{k,n}(u)$$

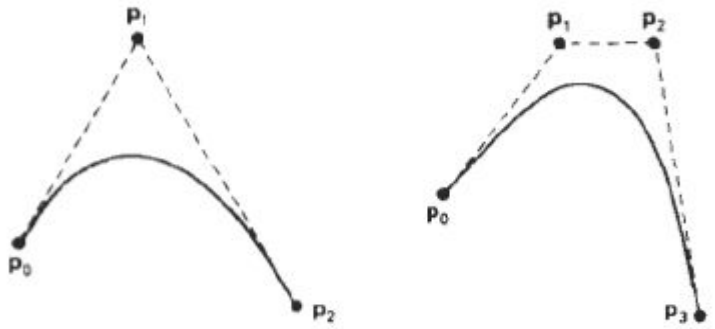


Fig: Bezier curve generated by three and four control point.

Cubic Bezier Curve

- It is a Bezier curve generated by four control points.
- General equation for cubic Bezier curve is

$$P(u) = \sum_{k=0}^{3} p_k BEZ_{k,n}(u), \qquad 0 \le u \le 1 \quad \dots\dots\dots\dots (i)$$

$$P(u) = p_0 BEZ_{0,3}(u) + p_1 BEZ_{1,3}(u) + p_2 BEZ_{2,3}(u) + p_3 BEZ_{3,3}(u)$$

Where,

$$BEZ_{0,3}(u) = C(3,0)u^0(1-u)^{3-0} = \frac{3!}{0!\,(3-0)!} \times (1-u)^3 = (1-u)^3$$

Similarly,

$$BEZ_{1,3}(u) = 3u(1-u)^2$$
$$BEZ_{2,3}(u) = 3u^2(1-u)$$
$$BEZ_{3,3}(u) = u^3$$

$$\boxed{\therefore \; P(u) = p_0(1-u)^3 + p_1 3u(1-u)^2 + p_2 3u^2(1-u) + p_3 u^3}$$

In matrix form,

$$P(u) = [u^3 \quad u^2 \quad u \quad 1].M_{BEZ}.\begin{bmatrix} p_0 \\ p_1 \\ p_2 \\ p_3 \end{bmatrix}$$

Where,

$$M_{BEZ} = Bezier\ matrix = \begin{bmatrix} -1 & 3 & -3 & 1 \\ 3 & -6 & 3 & 0 \\ -3 & 3 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

Properties of Bezier curve:

a) The basis functions are real.
b) The Bezier curve always passes through first and last control point i.e. $p(0) = p_0$ & $p(1) = p_n$.
c) The degree of polynomial representing Bezier curve is one less than the number of control points.
d) The Bezier curve always follows convex hull formed by control points.
e) The Bezier curve always lies inside the polygon formed by control points.
f) Bezier blending functions are positive and sum is equal to 1.
$\sum_{k=0}^{n} BEZ_{k,n}(u) = 1$
g) The direction of the tangent vector at the end points is same like vector determined by first and last segment.

Construct the Bezier curve of order 3 with 4 vertices of the control polygon $p0(0, 0), p1(1, 2), p2(3, 2)$& $p3(2, 0)$. Generate at least 5 points on the curve.

Solution:

Given control points $p0 = (x0, y0) = (0,0)$

$p1 = (x1, y1) = (1,2)$

$p2 = (x2, y2) = (3,2)$

$p3 = (x3, y3) = (2,0)$

Here, degree n= 3

We have basis function as

$$P(u) = \sum_{k=0}^{n} p_k BEZ_{k,n}(u), \qquad 0 \le u \le 1$$

$$= \sum_{k=0}^{3} p_k BEZ_{k,3}(u), \qquad 0 \le u \le 1$$

$$\therefore\ P(u) = p_0 BEZ_{0,3}(u) + p_1 BEZ_{1,3}(u) + p_2 BEZ_{2,3}(u) + p_3 BEZ_{3,3}(u)$$

Parametric equations are;

$$x(u) = x_0 BEZ_{0,3}(u) + x_1 BEZ_{1,3}(u) + x_2 BEZ_{2,3}(u) + x_3 BEZ_{3,3}(u) \ \ldots\ldots\ldots\ldots \text{(i)}$$
$$y(u) = y_0 BEZ_{0,3}(u) + y_1 BEZ_{1,3}(u) + y_2 BEZ_{2,3}(u) + y_3 BEZ_{3,3}(u) \ \ldots\ldots\ldots\ldots \text{(ii)}$$

Now,

$$BEZ_{0,3}(u) = C(3,0)u^0(1-u)^{3-0} = \frac{3!}{0!(3-0)!} \times (1-u)^3 = (1-u)^3$$
$$BEZ_{1,3}(u) = C(3,1)u^1(1-u)^{3-1} = \frac{3!}{1!(3-1)!} \times u(1-u)^2 = 3u(1-u)^2$$
$$BEZ_{2,3}(u) = C(3,2)u^2(1-u)^{3-2} = \frac{3!}{2!(3-2)!} \times u^2(1-u)^1 = 3u^2(1-u)$$
$$BEZ_{3,3}(u) = C(3,3)u^3(1-u)^{3-3} = \frac{3!}{3!(3-3)!} \times u^3(1-u)^0 = u^3$$

Putting these values in eq. (i) & (ii) we get;

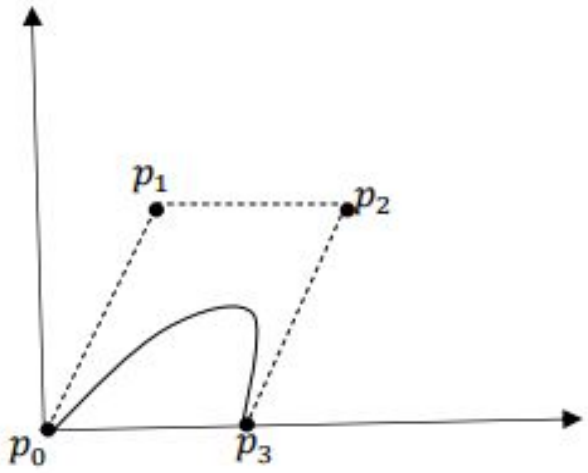
$$x(u) = x_0(1-u)^3 + x_1 3u(1-u)^2 + x_2 3u^2(1-u) + x_3 u^3 = -4u^3 + 3u^2 + 3u$$

$$y(u) = y_0(1-u)^3 + y_1 3u(1-u)^2 + y_2 3u^2(1-u) + y_3 u^3 = -6u^2 + 6u$$

Now,

	$x(u)$	$y(u)$
u =0	0	0
u =0.15	0.50	0.76
u =0.35	1.24	1.36
u =0.5	1.75	1.50
u =0.65	2.12	1.36
u =0.85	2.14	0.76
u =1	2	0

Plotting the graph;



Applications of Bezier Curves-

Bezier curves have their applications in the following fields-

1. Computer Graphics-

Bezier curves are widely used in computer graphics to model smooth curves.

The curve is completely contained in the convex hull of its control points.

So, the points can be graphically displayed & used to manipulate the curve intuitively.

2. Animation-

Bezier curves are used to outline movement in animation applications such as Adobe Flash and synfig.

Users outline the wanted path in bezier curves.

The application creates the needed frames for the object to move along the path.

For 3D animation, bezier curves are often used to define 3D paths as well as 2D curves.

3. Fonts-

True type fonts use composite bezier curves composed of quadratic bezier curves.

Modern imaging systems like postscript, asymptote etc use composite bezier curves composed of cubic bezier curves for drawing curved shapes.

Bezier surfaces

- A Bezier surface is an extension of a Bezier curve into **two dimensions.**

- Used to create **smooth 3D surfaces.**

- Defined by a **grid (mesh) of control points and the surface is shaped according to these control points.**

- Commonly used in **3D graphics, CAD, and modeling software.**

Properties of Bezier Surfaces: Bezier surfaces share many properties with Bezier curves, including:

- They pass through the corner (first and last) control points.

- The surface always lies within the convex hull of its control points.

- The blending functions used to construct the surface are positive.
  These properties make Bezier surfaces particularly useful for designing complex objects such as car bodies, furniture, and other products that require smooth, flowing shapes.

Equation for the Bezier surface is quite similar to the curve –

$$P(u,v) = \sum_{i=0}^{n}\sum_{j=0}^{m} p_{i,j}\, BEZ_{i,n}(u)\, BEZ_{j,m}(v), \quad 0 \le u, v \le 1$$

Where,

- $p_{i,j}$ represents the control points.
- $BEZ_{i,n}(u)$ and $BEZ_{j,m}(v)$ are the blending function.
- u,v are parameters that varies between 0 and 1, controlling the position along the curve.
- $BEZ_{i,n}(u) = C(n,i)u^i(1-u)^{n-i}$
- $BEZ_{j,m}(v) = C(m,j)v^j(1-v)^{m-j}$

How Bezier Surfaces Work?

- Control points influence the **shape of the surface.**
- The surface does **not need to pass through all control points.**
- Moving a control point smoothly changes the surface.
- Uses **Bezier blending (Bernstein) functions** in two directions.

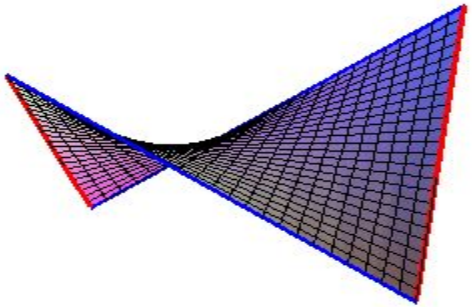
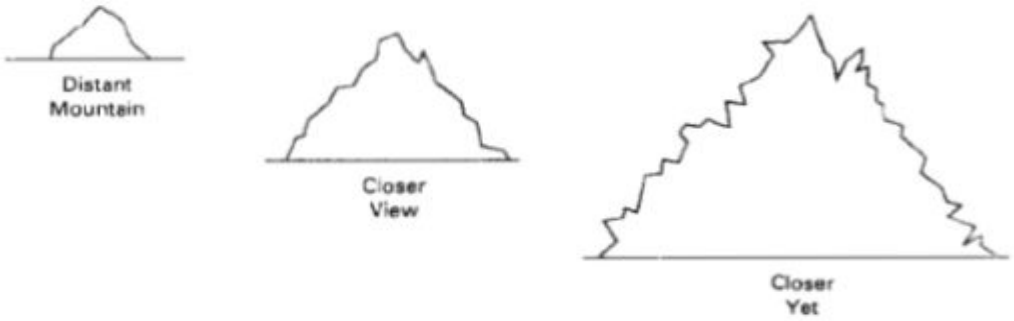


figure : bezier surface

Fractal Geometry Methods:

- Fractal objects refer to those objects which are self similar at all resolutions.
- Most of the natural objects such as trees,mountains and coastlines are considered as fractal objects because no matter how far or how close one looks at them, they appear to be somewhat similar. Which means it is the same from near as fram far.
- Natural objects,such as mountains and clouds, have irregular or fragmented features,and these are described with fractal geometry methods .
- Natural objects, such as mountains and clouds, don’t have smooth surface or regular shapes instead they have fragmented features, and Euclidean methods do not realistically model these objects.
- Natural objects can be realistically described with fractal-geometry methods, where **procedures rather than equations** are used to model objects.
- In computer graphics, fractal methods are used to generate displays of natural objects and visualizations of various mathematical and physical systems.
- A fractal object has t**wo basic characteristics**: **infinite detail at every point and certain self similarity between the object parts .**

- We describe a fractal object with a procedure that specifies a repeated operation for producing the detail in the object subparts. Natural objects are represented with procedures that theoretically repeat an infinite number of times. Graphics displays of natural objects are, of course, generated with a finite number of steps.
- If we zoom in on a continuous Euclidean shape, no matter how complicated, we can eventually get the zoomed-in view to smooth out.
- But if we zoom in on a fractal object, we continue to see as much detail in the magnification as we did in the original view.



- Zooming in on a graphics display of a fractal object is obtained by selecting a smaller window and repeating the fractal procedures to generate the detail in the new window.
- A consequence of the infinite detail of a fractal object is that it has no definite size. As we consider more and more detail, the size of an object tends to infinity, but the coordinate extents of the object remain bound within a finite region of space.
- The amount of variation in the object detail with a number called the **fractal dimension.**
- In graphics applications, fractal representations are used to model terrain, clouds, water, trees and other plants,feathers, fur, and various surface textures, and just to make pretty patterns.

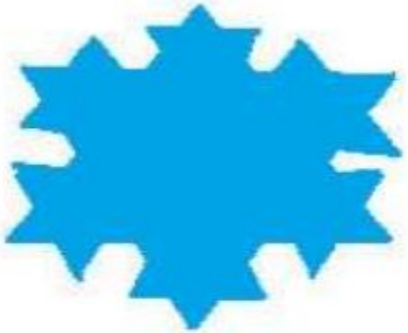
Generation of Fractals

Fractals can be generated by repeating the same shape over and over again as shown in the following figure. In figure (a) shows an equilateral triangle. In figure (b), we can see that the triangle is repeated to create a star-like shape. In figure (c), we can see that the star shape in figure (b) is repeated again and again to create a new shape.

We can do unlimited number of iteration to create a desired shape. In programming terms, recursion is used to create such shapes.

(a) Zeroth Generation

(b) First Generation



(c) Second Generation

Fractal generation procedure :

- A fractal is created by **repeating the same mathematical operation again and again** on points or shapes. We start with an **initial point or shape**, and each repetition adds **more detail** to it.
- The transformation can be applied to,Points,Lines ,Curves,Surfaces,Solid objects.
- The process can be:**Deterministic** (same result every time), or **Random** (slightly different results each time).
- Transformations may involve:Scaling (resizing),Rotation,Shifting (translation),More complex, non-linear coordinate transformation.
- Although fractals are **theoretically infinite in detail**, we apply the transformation **only a limited number of times**.Because of this, the fractals shown on a screen have **finite size and detail**.
- More iterations produce **more detailed and realistic fractals**.
- The amount of detail included in the final graphical display of an object depends on the number of iterations performed and the resolution of the display system
- We cannot display detail variations that are smaller than the size of a pixel.
- To see more of the object detail, we zoom in on selected sections and repeat the transformation function iterations.

Types of Fractal

There are three types of fractals:

- **Self-similar:** These fractals have parts as a scaled-down version of the entire object. So if we scale up from a smaller part of the fractal, we will scale up to the whole object.
- **Self Affine:** It has parts formed with different scaling parameters in different coordinate directions. For example in the direction, the scaling may be different from that t of y-direction. So the fractal will not be exactly similar in all directions.
- **Invariant:** Formed with nonlinear transformations. The smaller and bigger objects in the fractal are different.

Fractals are used in many areas such as –

Astronomy – For analyzing galaxies, rings of Saturn, etc.

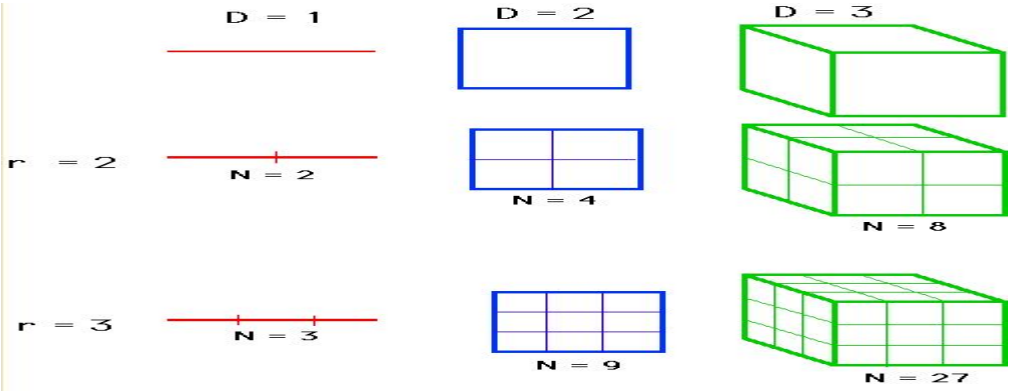
Biology/Chemistry – For depicting bacteria cultures, Chemical reactions, human anatomy, molecules, plants,

Others – For depicting clouds, coastline and borderlines, data compression, diffusion, economy, fractal art, fractal music, landscapes, special effect, etc.

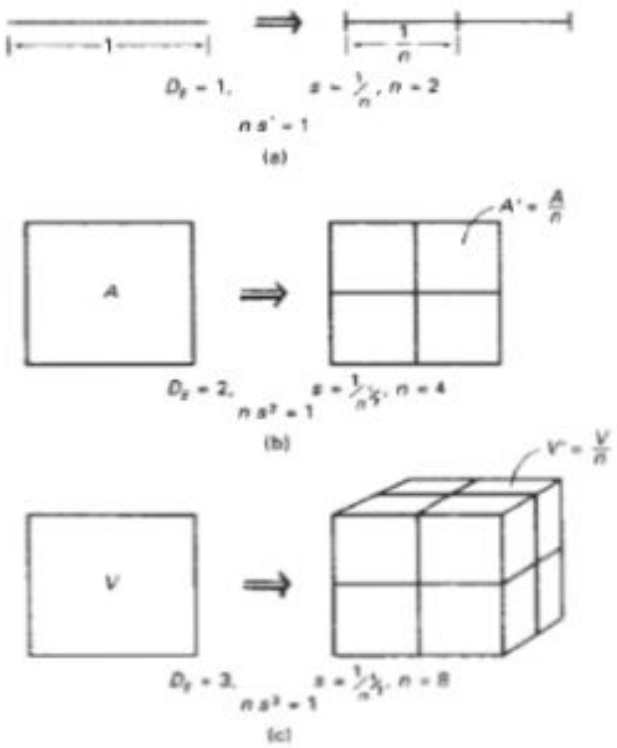
Fractal dimension:

- Fractal dimension is a measure of how complicated a self similar figure is.
- The detail variation in a fractal object can be described with a number D, called the fractal dimension, which is a measure of the roughness, or fragmentation, of the object.
- tells us **how complicated a pattern is** as we look closer and closer.
- Unlike regular shapes (like lines or squares), fractals are so detailed that normal dimensions aren't enough to describe them.

Dimensions of regular shapes



Suppose an object is composed of clay or elastic. If it is deformed into a line then its topological dimension D_t is 1, if it is deformed into a plane or a disk then the topological Dimension is 2 and if it is deformed into a ball or a cube then its topological dimension is 3



- The relationships between the scaling factor s; and the number of subparts n for subdivision of a unit straight-line segment, A square, and a cube can be shown.
- If take a line segment having length L and divide it into n pieces each piece having length ‘l’
- The scaling factor s = 1/n.If it is broken into two pieces, s1 = 1/2, the unit line segment is divided into two equal-length subparts.
- Similarly, the square is divided into four equal-area subparts, s2 = 1/4
- The cube is divided into eight equal-volume subparts s3 = 1/8
- For each of these objects, the relationship between the number of subparts and the scaling factor is

$$n \cdot s^D = 1.$$

Solving this expression for D, the fractal similarity dimension, we have

$$D = (\log n) / (\log (1/s))$$

The fractal dimension gives the measure of the roughness or fragmentation of objects and is always greater than the corresponding topological dimension.

Geometric Construction of Deterministic Self-Similar Fractals:

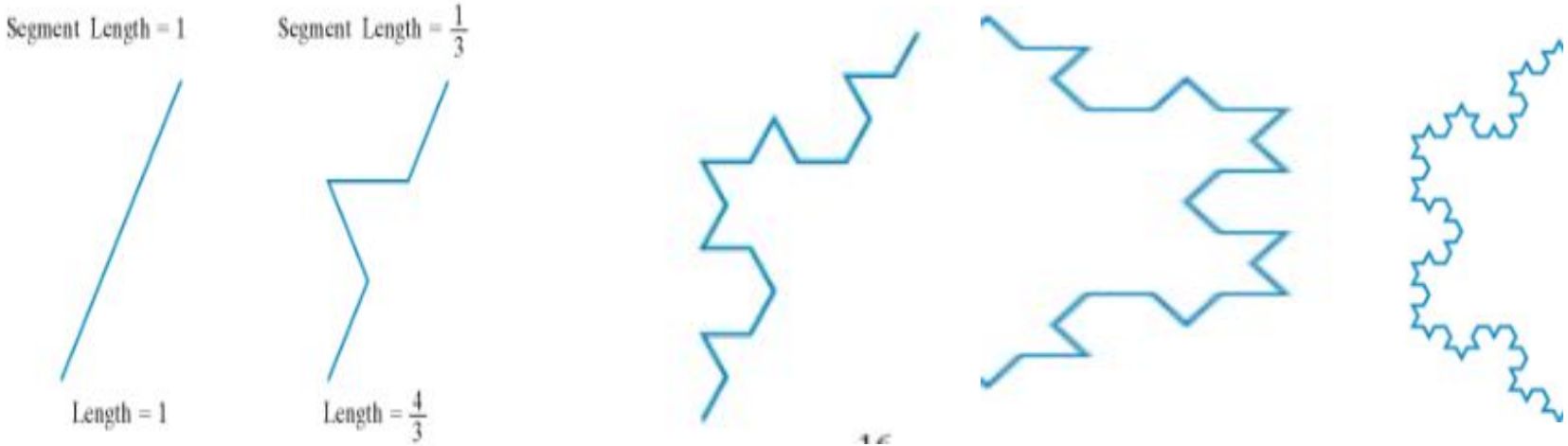
To geometrically construct a deterministic (nonrandom) self-similar fractal, we start with a given geometric shape, called the initiator. Subparts of the initiator are then replaced with a pattern, called the generator.

Koch Curve:

Step1: divide a straight line in three equal parts.

step2 : place equilateral triangle on middle part and remove the base of middle part . now the curve has four segments.

step 3: Divide each segment into 3 equal part and repeat step2



- This section goes for infinite times,if we zoom any section of the curve the basic triangle structure can be seen.

Clipping in 3D

- 3D clipping, in graphics technology is used to speed up your rendering engine. The most common clipping method is the 2 D clipping algorithm.
- 2D clipping is done at the last stage of rendering.
- The triangle routine used to render the polygons onto the screen make sure that you don’t draw outside the screen.
- This can be very fast,but increases the complexity of the triangle-filler.
- The clipping method works directly with two-dimensional screen- coordinates.
- In 3D clipping you do everything in 3D space.
- Clipping in 3 dimensions can be accomplished using extensions of two D clipping methods.
- An algorithm for 3-D clipping identifies and saves all surface segments within the view volume for display on the output device.
- All part of objects that are outside the view volume are discarded.
- To clip a polygon surface ,we can clip the individual polygon edges.
- To clip a line segment against the view volume ,we would need to test the relative position of the line using the view volume boundary plane equations.
- An endpoint(x,y,z) of a line segments

- Outside a boundary plane: $Ax+By+Cz+D>0$,where A,B,C and D are the plane parameters for that boundary.
- Inside the boundary if $Ax+By+Cz+D<0$
- Lines with both endpoints outside a boundary plane are discarded , and those with both endpoints inside boundary planes are saved.